



NORMALIZACIÓN DE BASES DE DATOS



1FN · 2FN · 3FN — organizar los datos para reducir redundancia y evitar anomalías



Estudiante: Eduardo Coronado Santana

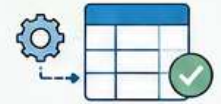


Grupo: 3DSM-G2



¿QUÉ ES LA NORMALIZACIÓN?

Es un proceso de diseño de bases de datos que organiza la información en tablas relacionadas. Su objetivo es reducir datos repetidos y prevenir problemas al insertar, actualizar o eliminar información. La normalización se realiza mediante formas normales; aquí se presentan la 1FN, 2FN y 3FN.



1FN · PRIMERA FORMA NORMAL

Regla clave: cada campo debe contener un solo valor (valor atómico) y no debe haber grupos repetitivos.

Idea visual: una celda = un dato.

ANTES



AlumnoID	Alumno	Teléfonos
1	Ana Pérez	5551, 5552

DEPUÉS

Alumno_Teléfono

AlumnoID	Teléfono
1	5551
1	5552



2FN · SEGUNDA FORMA NORMAL

Regla clave: estar en 1FN y eliminar dependencias parciales: cada atributo que no sea clave debe depender de toda la clave primaria.

Ejemplo: en una tabla con clave compuesta (AlumnoID, CursoID), el nombre del alumno depende solo de AlumnoID. Se separa en una tabla ALUMNO y otra INSCRIPCIÓN.

INSCRIPCIÓN (antes)

AlumnoID	CursoID	NombreAlumno	Curso
1	10	Ana Pérez	Bases de Datos
1	20	Ana Pérez	Programación



Después (en 2FN)

AlumnoID	Nombre
1	Ana Pérez

INSCRIPCIÓN

AlumnoID	CursoID
1	10
1	20



3FN · TERCERA FORMA NORMAL

Regla clave: estar en 2FN y eliminar dependencias transitivas: los atributos no clave deben depender directamente de la clave, no de otro atributo no clave.

Ejemplo: AlumnoID → CódigoPostal → Ciudad. La ciudad no debe depender indirectamente de AlumnoID; se puede crear una tabla CODIGO_POSTAL.

ALUMNO (2FN)

AlumnoID	Nombre	CódigoPostal	Ciudad
1	Ana Pérez	42080	Pachuca



Después (en 3FN)

ALUMNO

AlumnoID	Nombre	CódigoPostal
1	Ana Pérez	42080

CODIGO_POSTAL

CódigoPostal	Ciudad
42080	Pachuca



EJEMPLO APLICADO · DE UNA TABLA REPETITIVA A UN DISEÑO NORMALIZADO

TABLA VENTAS (NO NORMALIZADA)

VentaID	Cliente	Productos
101	Ana	Teclado, Mouse
102	Luis	Monitor, Cable



DISEÑO NORMALIZADO

CLIENTE: ClienteID, Nombre

ClienteID	Nombre
1	Ana
2	Luis

VENTA: VentaID, ClienteID

VentaID	ClienteID
101	1
102	2

PRODUCTO: ProductoID, Nombre

ProductoID	Nombre
1	Teclado
2	Mouse
3	Monitor
4	Cable

DETALLE_VENTA: VentaID, ProductoID, Cantidad

VentaID	ProductoID	Cantidad
101	1	1
101	2	1
102	3	1
102	4	1

CÓDIGO SQL (EJEMPLO)

```
-- CLIENTE
CREATE TABLE Cliente (
  ClienteID SERIAL PRIMARY KEY,
  Nombre VARCHAR(100) NOT NULL
);

-- VENTA
CREATE TABLE Venta (
  VentaID SERIAL PRIMARY KEY,
  ClienteID INT NOT NULL,
  FOREIGN KEY (ClienteID) REFERENCES Cliente(ClienteID)
);

-- PRODUCTO
CREATE TABLE Producto (
  ProductoID SERIAL PRIMARY KEY,
  Nombre VARCHAR(100) NOT NULL
);

-- DETALLE_VENTA
CREATE TABLE Detalle_Venta (
  VentaID INT NOT NULL,
  ProductoID INT NOT NULL,
  Cantidad INT NOT NULL,
  PRIMARY KEY (VentaID, ProductoID),
  FOREIGN KEY (VentaID) REFERENCES Venta(VentaID),
  FOREIGN KEY (ProductoID) REFERENCES Producto(ProductoID)
);
```



¿POR QUÉ ES IMPORTANTE?



Reduce redundancia

Evita guardar el mismo dato innecesariamente.



Evita anomalías

Disminuye errores de inserción, actualización y eliminación.



Mejora el mantenimiento

Los datos quedan organizados y sus relaciones son más claras.

REGLA RÁPIDA PARA RECORDAR

1FN

datos atómicos

2FN

sin dependencias parciales

3FN

sin dependencias transitivas



CONCLUSIÓN

Normalizar permite construir bases de datos más consistentes, ordenadas y fáciles de mantener. Al avanzar de 1FN a 3FN se eliminan diferentes tipos de redundancia y dependencias inadecuadas.



FUENTES CONSULTADAS

- PostgreSQL Documentation – Database Design. <https://www.postgresql.org/docs/>
- MySQL 8.0 Reference Manual – Database Normalization. <https://dev.mysql.com/doc/>
- Microsoft Learn – Normalización de bases de datos. <https://learn.microsoft.com/>



Actividad basada en las indicaciones del archivo "Actividad de Evaluación: Infografías de Bases de Datos y actualización de repositorio GitHub."